23CSE452 Business Analytics

Project

Movie Performance Analysis and Prediction Using TMDB Data

Submitted in partial fulfillment of the Completion of
the course **23CSE452 Business Analytics**
under the **Faculty of Computer Science and Engineering**

by

NAME	ROLL NUMBER
Soundarya Satalgoan	CB.SC.U4CSE23447
Balaji N	CB.SC.U4CSE23011
Parvathy Krishna A	CB.SC.U4CSE23739
Venkata Kanna Bhavan Surya Adapa	CB.SC.U4CSE23467
Dareddy Tejeswara Reddy	CB.SC.U4CSE23614



Deemed to be University under Section 3 of the UGC Act, 1956
Department of Computer Science and Engineering

AMRITA VISHWA VIDYAPEETHAM

Coimbatore Campus (India)

November 2026

Team No: 06 | Programme: B.Tech CSE | Guide: Rohini S | Academic year: 2023-2027

Declaration and Acknowledgement

Declaration. This report is our own work, prepared for the 23CSE452 Business Analytics course. The notebooks are ours too, and we wrote them together as a team. We built the dataset ourselves by collecting it from the TMDB API. Anything we borrowed (ideas, figures, code or data) is credited in the references. We used a generative-AI assistant, and Section 8.3 explains how. We have not copied from any other team.

Name	Register number	Signature
Soundarya Satalgoan	CB.SC.U4CSE23447	
Balaji N	CB.SC.U4CSE23011	
Parvathy Krishna A	CB.SC.U4CSE23739	
Venkata Kanna Bhavan Surya Adapa	CB.SC.U4CSE23467	
Dareddy Tejeswara Reddy	CB.SC.U4CSE23614	

Acknowledgement. We thank our course faculty, Rohini S, for guiding us through the reviews, and The Movie Database (TMDB) for giving us API access to collect our data. This product uses the TMDB API but is not endorsed or certified by TMDB.

Abstract

Audience ratings depend on period, language and genre, and studios and streaming platforms keep struggling to decide which films to commission, buy or promote. We built our own dataset by collecting movies from the TMDB API, taking random pages within each release year from 2000 to 2025 so that popular titles would not dominate. That gave 15,555 movies, and 15,386 remained after cleaning. Of these, 10,078 had 20 or more votes, which is the cut-off we use for every rating analysis. Genre is the strongest signal (Documentary 6.89, Animation 6.82, Horror 5.36), while the numeric variables correlate only weakly with rating (0.11 to 0.23). A Random Forest that sees only the movie's content, with no early votes or popularity, predicts whether a movie is high-rated (6.5 or above) with a test AUC of 0.785, against 0.50 for a majority-class baseline. Adding early votes and popularity lifts the test AUC to 0.836. For Unit 3 we used two methods. In the time-series work, the monthly average rating stayed near 6.06 until 2019 and near 6.31 since 2020, with little seasonality and almost no gain over a naive baseline. In the text mining of plot summaries, 19 words were significantly linked to rating, and the four topics differed in average rating from 6.01 to 6.31. Everything is brought together in a Streamlit dashboard, and we finish with recommendations for genre planning and early screening.

Keywords: movie ratings, TMDB, predictive modelling, time-series forecasting, text mining, topic modelling, dashboard

Executive Summary

Problem. Content teams at studios and streaming services want to know what kinds of films tend to do well, both before release and once the first audience response comes in. We studied the ratings of movies from 2000 to 2025, collected from The Movie Database.

What we found. Genre matters most. Documentary and Animation films rate about 6.8 to 6.9 out of ten, while Horror sits at 5.4. A model that uses only content (genre, language, year) picks out higher-rated movies much better than chance, with an AUC of 0.79 against 0.50. With early votes and popularity added, the AUC reaches 0.84. Even so, the model misses around 43% of the higher-rated movies. The average rating stayed near 6.06 until 2019 and has been around 6.3 since 2020, with little seasonal change, so we would assume it stays level rather than expect it to climb. Horror and Thriller are the two lowest-rated genres, and both gained share, while Drama lost share. Words like documentary, war and history show up in the plots of higher-rated movies. Killer, couple and boyfriend show up in lower-rated ones.

What to do. (1) Compare each title with the typical rating in its genre. (2) Use the content-only model as a first screen: 72% of the titles it flags are high-rated, against 39% by chance. (3) Watch early votes once a title is released. (4) Add a quality-review step for the growing Horror and Thriller slate. (5) Compare pitches with the ratings of films that have similar plots. These results come from a sample of the TMDB database and show associations, not causes. The model should be one input to a decision, never the only one. The interactive dashboard lets management explore everything by year, genre and language.

Table of Contents

Abstract ..... **i**
Executive Summary ..... **ii**
1. Introduction, Business Objectives and Literature Review **1**
1.1 Context and scope 1
1.2 Related analytics work and research gap 1
2. Dataset Collection, Validation and Preparation **2**
2.1 Provenance, coverage and challenges 2
2.2 Data quality, privacy and ethics 3
2.3 Transformations and usable dataset 3
3. Exploratory Data Analysis and Visualization (Unit 1) **4**
4. Feature Engineering and Predictive Modelling (Unit 2) **5**
5. Advanced Analytics (Unit 3): Two Selected Methods **7**
5.1 Method 1: Time-series analysis 7
5.2 Method 2: Text mining of plot overviews 9
6. Comparative Analysis and Business Impact **10**
6.1 Comparison of the two Unit 3 methods 10
6.2 Comparison with existing studies 11
6.3 KPI-based business impact and scenarios 11
7. Interactive Application, Figure Index and Reproducibility **12**
7.1 Mandatory dashboard 12
7.2 Figure index and reproduction map 12
7.3 Code and environment 13
8. End-to-End Validation, Traceability and Contributions **13**
8.1 Completion and validity 13
8.2 Project tracking and individual work 13
8.3 AI use and originality declarations 14
9. Conclusion, Recommendations and Roadmap **14**
References **15**
Appendix A: Extended Data Dictionary and Validation Records **15**
Appendix B: Additional Figures and Supplementary Analysis **16**
Appendix C: Implementation Details and Dashboard User Guide **16**
Appendix D: Tracking Evidence, Contributions and Declarations **17**

List of Figures

Figure 1. End-to-end workflow connecting data collection, Units 1 to 3 and the dashboard....1
Figure 2. Average rating and popularity by genre and by language....4
Figure 3. Rating by decade and yearly average rating and popularity....5
Figure 4. Test AUC by model and feature set....5
Figure 5. Confusion matrices and ROC curves on the test set....6
Figure 6. Permutation importance of the top 12 features....7
Figure 7. Monthly average rating, and its additive decomposition....8
Figure 8. Share of sampled movies in the six largest genres by year....8
Figure 9. Forecast against actual (2023 to 2024) and forecast for 2025....8
Figure 10. Words significantly linked to rating....9
Figure 11. Average rating and high-rated share by dominant topic....10
Figure 12. Sentiment of plot summaries....10
Figure 13. Dashboard main and filtered states....12

List of Tables

Table 1. Business objectives, KPIs and success criteria....1
Table 2. Dataset provenance and size....2
Table 3. Sample rows from the final dataset....2
Table 4. Data quality checks, findings and corrections....3
Table 5. Final dataset fields and where they are used....3
Table 6. Model comparison on the held-out test set....6
Table 7. Forecast evaluation on the last 24 months....8
Table 8. LDA topics with top words, size and rating....9
Table 9. Comparison of time-series analysis and text mining....10
Table 10. Current project compared with prior studies....11
Table 11. Scenario: picking 285 of 1,000 candidate titles....11
Table 12. Figure index....12
Table 13. Objective-to-evidence traceability matrix....13
Table 14. Team members, assigned work and evidence....14
Table 15. Prioritised recommendations....14
Table 16. Movies per genre and per language....16
Table 17. Key settings used....16

List of Abbreviations

ADF	Augmented Dickey-Fuller test	**LDA**	Latent Dirichlet allocation
AIC	Akaike information criterion	**MAE / RMSE**	Mean absolute / root mean squared error
API	Application programming interface	**NPMI**	Normalised pointwise mutual information
AUC	Area under the ROC curve	**ROC**	Receiver operating characteristic
CV	Cross-validation	**SARIMA**	Seasonal autoregressive integrated moving average
EDA	Exploratory data analysis	**TF-IDF**	Term frequency, inverse document frequency
IQR	Interquartile range	**TMDB**	The Movie Database
KPI	Key performance indicator		

1. Introduction, Business Objectives and Literature Review

1.1 Context and scope

Every year, studios and streaming companies must decide which films to produce, buy or promote. Audience ratings are one of the best signs of how a film lands with the public, but ratings vary by genre, language and time, and it is not obvious which kinds of films do well. We looked at this question for movies from 2000 to 2025, using data our team collected from the TMDB API [1]. The readers we have in mind are content acquisition and programming teams who need a realistic expectation of ratings before and after release. Only ratings given by TMDB audiences are considered. Revenue, budget and cast effects are out of scope, and everything we report is an association, not a cause.

We want to find which attributes go with high ratings and to predict whether a given movie will get a high rating, which here means 6.5 or above. Four questions guide the work: (Q1) which genres, languages and periods have higher ratings; (Q2) whether high-rated movies can be predicted before or after release; (Q3) how ratings change over time and what level to expect; and (Q4) which words and themes in plot summaries go with high or low ratings. Figure 1 shows how the three units and the dashboard connect, and Table 1 lists the measurable KPIs.

Table 1. Business objectives, KPIs and success criteria.

Business objective	KPI (unit)	Baseline / target	Unit or method
O1. Build an independent, non-popularity-biased dataset	Clean records (count)	Target: at least 10,000. Achieved: 15,386 (10,078 for modelling)	Unit 1
O2. Predict high-rated movies	Test AUC (0 to 1)	Baseline 0.50 (majority class). Target: at least 0.75	Unit 2
O3. Forecast the rating level	RMSE (rating points)	Baseline: naive RMSE 0.173. Target: lower than naive	Unit 3, Method 1
O4. Find themes linked to ratings	Tested word and topic differences (p-value, effect size)	Target: interpretable topics with a tested rating difference	Unit 3, Method 2
O5. Deliver an interactive application	Working filters, all units integrated	Target: dashboard with filters over Units 1 to 3	Application

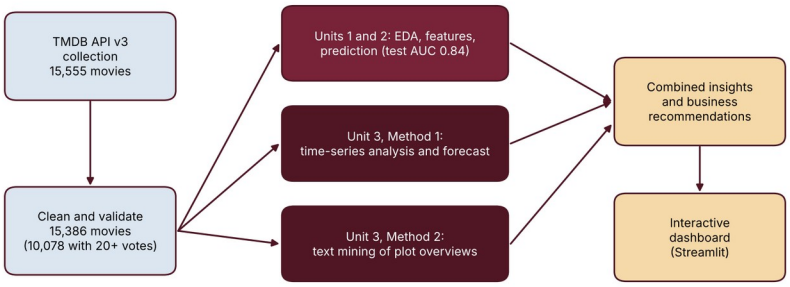


Figure 1. End-to-end workflow connecting data collection, Units 1 to 3 and the dashboard. Tool: Python (Matplotlib). Source: reports/figures/workflow.

1.2 Related analytics work and research gap

Most research on movie success looks at box-office results. Sharda and Delen [2] used a neural network to classify movies into box-office categories from pre-release attributes such as genre, star value, sequel status and number of screens. Asur and Huberman [3] used the volume and sentiment of tweets to predict opening-

weekend revenue. Public datasets such as MovieLens and the Kaggle TMDB 5000 file are fixed snapshots, and our course did not allow them as a source.

Two gaps matter for our setting. First, revenue studies need budget, screen-count or social-media data, which TMDB does not offer, and audience rating is the only outcome we really have. Second, most studies do not separate predicting from content (before release) from predicting from audience activity (after release), and they do not connect rating trends and plot text on the same dataset. We therefore apply classification (Unit 2), time-series analysis and text mining (Unit 3) to one dataset we collected ourselves. Section 6.2 compares our results with the earlier studies.

2. Dataset Collection, Validation and Preparation

2.1 Provenance, coverage and challenges

We collected the data ourselves from the TMDB API v3 [1], using /discover/movie for movies and /genre/movie/list for genre names. No existing dataset was used. TMDB limits each query to 500 pages, and sorting by popularity would fill a sample with well-known films. So we ran one query per year from 2000 to 2025 and took 30 random pages from each (seed 42). Each query was sorted by popularity, required at least 10 votes and left out adult content. Random pages give a mix of hits and obscure titles, around 600 titles per year. Table 2 summarises the source and size.

Table 2. Dataset provenance and size.

Item	Detail
Source and method	TMDB API v3; /discover/movie, /genre/movie/list - authorised API key stored locally and not in the repository as .env
Collection dates / version	Collected on 2026-10-01 ; TMDB API v3 (data change over time, so this is a snapshot)
Licence / permitted use	TMDB API terms: non-commercial use with attribution; used here for academic purposes only
Level of observation	One row for each movie (key: TMDB id)
Time period	Release dates 2000-01-01 to 2025-12-31 (575 to 598 movies per year after cleaning)
Records and variables	Raw: 15,555 movies x 9 variables. Clean: 15,386 x 11. Modelling subset (20 or more votes): 10,078
Target	high_rated = 1 if vote_average is 6.5 or more (39.0% of the modelling subset)

Collection did not go smoothly at first. The API kept dropping the connection (WinError 10054), so we switched to a single HTTP session with automatic reconnection and back-off, and saved a checkpoint after each year so a run could resume where it stopped. We also paused briefly between requests to stay within rate limits, and used the genre endpoint to turn the genre ID lists the API returns into names. Table 3 shows a few rows of the cleaned data. The full dictionary is in Table 5 and Appendix A.

Table 3. Sample rows from the final dataset (tmdb_clean.csv; overview text omitted).

id	title	release_date	lang.	genres	popularity	vote_average	vote_count	release_year
159627	Ground Zero	2000-05-12	en	Drama, Action	1.6756	4.1	13	2000
28085	Blind Target	2000-01-01	en	Thriller, Drama	1.7006	3.4	11	2000
60074	La Squale	2000-11-29	fr	Drama	1.8364	6.1	16	2000
262069	After-School Special	2011-06-23	en	Comedy, Drama	1.4963	5.0	15	2011
430181	Debbie Does Dallas Uncovered	2005-04-27	en	Documentary	1.5780	4.9	13	2005
57112	Have You Met Claudia?	2004-12-15	it	Comedy	2.2407	6.7	1072	2004
348514	Red Rover	2015-10-30	en	Drama, Science Fiction	0.9810	4.9	11	2015

2.2 Data quality, privacy and ethics

Table 4 lists the checks we ran on the raw data and what we kept. The cleaned data has no missing values and no duplicate IDs. The IQR rule on log values flagged 913 popularity outliers (5.9%) and 653 vote-count outliers (4.2%). These are genuine values, and removing them would have tilted the data toward obscure films, so we kept them.

Table 4. Data quality checks, findings and corrections.

Check	Finding	Action and records retained
Missing values	0 NaN values; 105 blank overviews	105 rows dropped (no text to analyse)
Duplicates	0 duplicate movie IDs	None needed
Invalid values	64 rows with no genre or invalid values (zero votes, popularity of 0 or less, rating outside 0 to 10)	64 rows dropped; final 15,386 rows (169 removed, 1.1%)
Dates	0 invalid and 0 future release dates	Parsed to date type
Anomalies	913 popularity and 653 vote-count outliers; 4 overviews under 5 words	Kept (genuine hits; negligible)
Imbalance	English 53%, 89 languages (top 10 = 84.9%); 34.5% of movies have under 20 votes	Top-10 language grouping; movies with 20 or more votes used for rating analysis and modelling

The data describe films, not the people who watch them, so no personal data is processed and the Digital Personal Data Protection Act, 2023 does not apply. The API key is kept out of the repository, and TMDB is credited in the README. Some biases are clear. Titles with 10 or more votes are still favoured even with random pages, English-language films are far more numerous, averages for movies with few votes are noisier, and we have only about 600 movies per year rather than the whole catalogue. Random-page sampling and the 20-vote rule reduce these effects, and the rest are covered in Section 8.1 (threats to validity).

2.3 Transformations and usable dataset

The same cleaning applies to all units. We removed duplicates, converted release_date to a date, dropped blank overviews and invalid rows, mapped genre IDs to names, and created two derived columns: release_year and low_votes (vote_count below 20). The row count went from 15,555 to 15,386. Of those, 10,078 have 20 or more votes and are used for modelling, and the other 5,308 are kept for exploration. Model-specific preprocessing came afterwards, after the train/test split: multi-hot genre flags, one-hot top-10 languages, log transforms of popularity and votes, a TF-IDF vectoriser fitted on training rows only, and scaling for logistic regression only. Table 5 maps each field to the units that use it.

Table 5. Final dataset fields and where they are used.

Variable	Type / meaning	Origin / preparation	Used in
id	Integer; TMDB movie ID	API; key for deduplication	All units (key)
title	Text; movie name	API	Dashboard explorer
release_date	Date	API; parsed to date	U1, U2 (month), U3 M1
original_language	Category; ISO code	API; top 10 kept, rest grouped as 'other'	U1, U2
overview	Text; plot summary	API; blanks removed; cleaned for text mining	U2 (word count), U3 M2
popularity	Number; TMDB activity score	API; log transform for models	U1, U2 (set B), U3 M1
vote_average	Number; mean rating 0 to 10	API; target source (6.5 or more)	U1, U2 (target), U3 M1, M2
vote_count	Integer; number of votes	API; 20-vote rule; log transform	U1, U2 (set B), filter
genres	List; 19 genre names	Mapped from genre IDs; multi-hot flags	U1, U2, U3 M1, M2
release_year	Integer	Derived from release_date	U1, U2, U3 M1
low_votes	Boolean flag	Derived: vote_count below 20	U1 (filter)

3. Exploratory Data Analysis and Visualization (Unit 1)

This section answers question 1 and sets up the features for Section 4. The rating distribution is roughly bell-shaped, with a median of 6.1, a mean of 6.00 and a slight left skew (-0.45). Popularity (skew 6.85) and vote count (skew 10.24) are strongly right-skewed, so log transforms make sense. The median popularity is 1.88 and the median vote count is 30, but the top titles reach a popularity of 92 and over 34,000 votes. We made the charts with Python (pandas, Matplotlib [4] and seaborn). For movies with 20 or more votes, the mean rises to 6.12 and the standard deviation falls to 0.99, which shows that part of the spread at the low end comes from small samples.

Genre separates ratings most clearly (Figure 2, movies with 20+ votes). Documentary (6.89), Animation (6.82), Music (6.65) and History (6.62) rate highest. Horror (5.36), Thriller (5.81) and Science Fiction (5.82) rate lowest. Rating and popularity do not move together in any simple way. Documentary has the highest rating but the lowest average popularity (2.05). Adventure is the most popular genre (6.67), and Animation is high on both measures (popularity 6.26). By language, English is second lowest on rating (5.99) and first on popularity (4.24), while Japanese (6.73) and Korean (6.65) average highest. Films with 20 votes are not a representative sample of world cinema, so this says nothing about the quality of films in other languages.

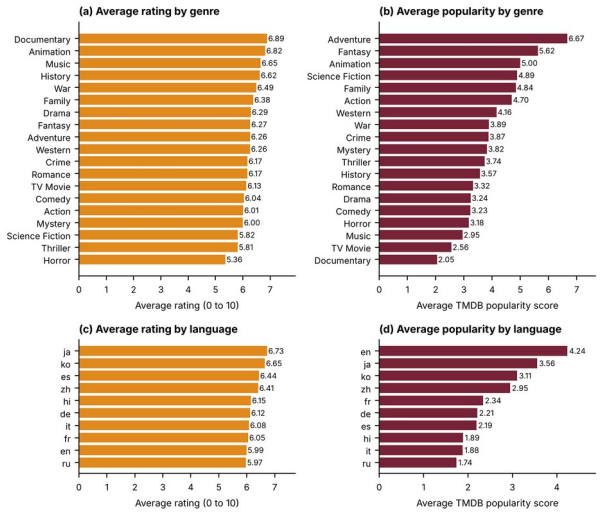


Figure 2. Average rating and popularity by genre (19 genres) and by language (10 most common), movies with 20 or more votes. Tool: Python (Matplotlib/Seaborn). Source: Review1_TMDB_Final.ipynb, EDA genre and language section.

Linear correlation with rating is weak. The Spearman correlations are 0.23 for log vote count, 0.16 for log popularity and 0.11 for release year (0.24, 0.16 and 0.08 when only movies with 20 or more votes are used). Vote count and popularity are correlated at 0.65 to 0.68. A density plot of rating against votes (Appendix B) shows ratings for low-vote movies spread widely and then settling around 6 to 7 as votes grow, which supports the 20-vote rule. By decade, ratings were flat in the 2000s (6.06, 3,827 movies) and the 2010s (6.06, 3,910) and higher in the 2020s (6.33, 2,341; Figure 3). This is only an association. TMDB popularity reflects recent activity, which favours newer films, and recent movies with 20 votes may be a better-received subset.

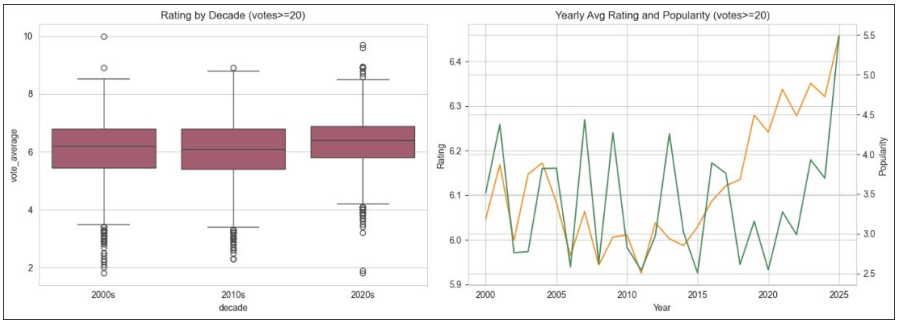


Figure 3. Rating by decade (box plot) and yearly average rating (orange) and popularity (green), movies with 20 or more votes. Tool: Python (Matplotlib/Seaborn). Source: Review1_TMDB_Final.ipynb, EDA relationships section.

These results shaped our choices: genre and language flags, log transforms for popularity and votes, the 20-vote rule for rating analysis, and the time-series analysis of the rating trend in Section 5.1.

4. Feature Engineering and Predictive Modelling (Unit 2)

The target is high-rated (6.5 or above), predicted for the 10,078 movies with at least 20 votes, of which 39.0% are positive. Because the numeric features are only weakly correlated with rating (Section 3), we chose classification over predicting the rating itself. To separate what is known before release from what is not, we built two feature sets. Set A (34 features, content only) holds release year and month, overview word count, number of genres, 19 genre flags, 10 language flags and an 'other' flag. Set B (36 features) adds log popularity and log vote count. We also tried TF-IDF text features (1,000 unigram and bigram terms). The feature count was small, so no dimensionality reduction was needed.

We used a stratified 80/20 split (8,062 movies for training, 2,016 for testing, 39% high-rated in both, seed 42). Models were tuned with 5-fold stratified cross-validation on AUC, using the training set only, and every model was scored on the same test movies. Leakage audit: the split happens before any model-specific processing; the TF-IDF vectoriser and scaler are fitted on training rows only; vote_average is never a feature; identifiers are not used; and no movie appears twice (id is unique). The model is cross-sectional, so time order does not matter.

A majority-class baseline gives an accuracy of 0.610 and an AUC of 0.50. Using scikit-learn [5], we trained Logistic Regression, Random Forest [6] and XGBoost [7] on four feature sets (Figure 4). The text features did not help. With 1,000 sparse features and only about 8,000 training rows, the models mostly fit noise, so we use text for explanation in Section 5.2 and not for prediction. Adding popularity and votes (set B) raises AUC by roughly 0.05 to 0.06.

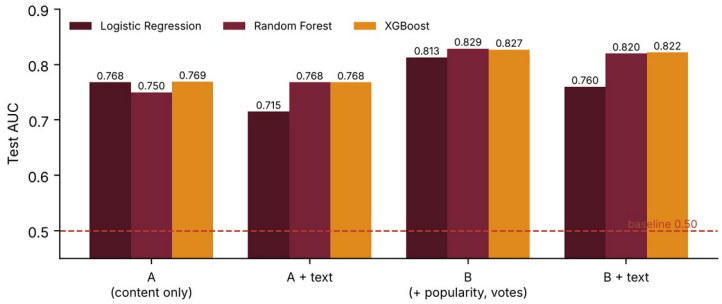


Figure 4. Test AUC by model and feature set. Dashed line: baseline AUC 0.50. Tool: Python (Matplotlib/Seaborn). Source: Review1_TMDB_Final.ipynb, model development.

We tuned the Random Forest and XGBoost hyperparameters with grid search and 5-fold cross-validation. Cross-validated AUC moved from 0.816 to 0.819 (Random Forest, set B; depth 20, minimum leaf 3), from

0.798 to 0.815 (XGBoost, set B; depth 3, 400 trees, learning rate 0.05), from 0.739 to 0.761 (Random Forest, set A) and from 0.728 to 0.757 (XGBoost, set A). The gains are small, and cross-validation and test scores agree closely, so overfitting does not look like a problem. We selected Random Forest on set B because of its cross-validated AUC (0.819). XGBoost is within noise on test AUC (0.834 against 0.836). Random Forest on set A is the pre-release model.

Table 6. Model comparison on the same held-out test set (2,016 movies). Precision, recall and F1 are for the high-rated class.

Model	Validation scheme	Test metrics	Interpretability / bias evidence
Baseline (majority class)	None	Accuracy 0.610; F1 0; AUC 0.500	Not applicable
Logistic Regression, set B	Scaler fitted on training data	Accuracy 0.763; F1 0.670; AUC 0.813	Coefficients available; not analysed
XGBoost, set B (tuned)	5-fold CV; CV AUC 0.815	Accuracy 0.771; F1 0.677; AUC 0.834	Permutation importance
Random Forest, set B (selected)	**5-fold CV; CV AUC 0.819**	**Accuracy 0.758; precision 0.747; recall 0.575; F1 0.650; AUC 0.836**	**Permutation importance (Figure 6)**
Random Forest, set A (content only)	5-fold CV; CV AUC 0.761	Accuracy 0.736; precision 0.720; recall 0.527; F1 0.608; AUC 0.785	Permutation importance (Figure 6)

Error analysis. For the chosen model (Figure 5), 1,077 movies were correctly classified as not high-rated, 153 were false alarms, and 334 high-rated movies were missed. The model plays it safe: when it says ‘high’, it is right 74.7% of the time, but it finds only 57.5% of the high-rated films. A lower threshold would catch more high-rated films at the cost of precision. With a 50-vote cut-off instead (5,717 movies), the AUC is 0.841, so the result does not depend on the 20-vote rule.

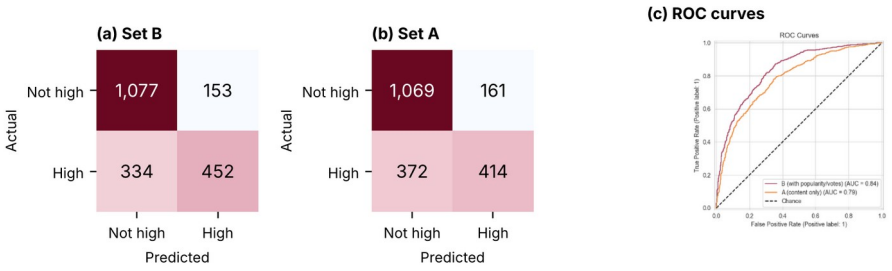


Figure 5. Confusion matrices for set B and set A, and ROC curves on the test set (AUC 0.84 and 0.79). Tool: Python (Matplotlib/Seaborn). Source: Review1_TMDB_Final.ipynb, model evaluation.

Interpretation. Permutation importance (Figure 6) shows log vote count as the most important feature in set B (0.066), followed by Documentary (0.049), Horror (0.032), English (0.031) and Animation (0.027). In set A the top five content signals are Documentary (0.049), Horror (0.038), Animation (0.037), release year (0.020) and Drama (0.016). This matches the EDA: genre is the strongest content signal. Release month and overview word count add almost nothing (about 0.003). Importance tells us what the model relies on, not the direction or cause of an effect. Set B uses post-release data, so it explains ratings and cannot predict a new release. We did not run subgroup analyses (for example by language or genre), so we do not know whether the model is biased across subgroups. This is a limitation (see Section 8.1).

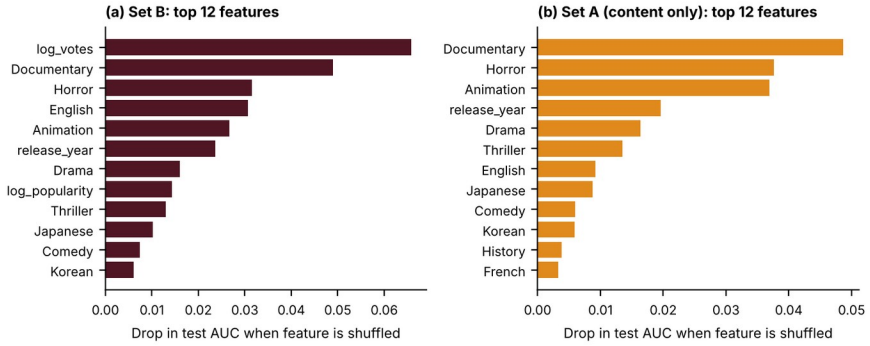


Figure 6. Permutation importance of the top 12 features for the Random Forest models (set B and set A). Tool: Python (Matplotlib/Seaborn). Source: Review1_TMDB_Final.ipynb, model evaluation.

The content-only model (AUC 0.785) works as a first screen before release, and the post-release model (AUC 0.836) refines expectations as early votes arrive. It misses too many high-rated titles to be the only deciding factor.

5. Advanced Analytics (Unit 3): Two Selected Methods

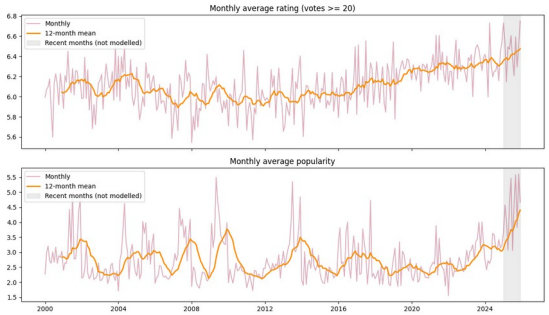
We considered three methods and chose two: time-series analysis (Method 1) and text mining (Method 2). Every movie has a release date and a plot summary, so these two fit the data naturally. We did not use association rule mining because our data are not transactions.

5.1 Method 1: Time-series analysis

Business question. What has changed in the average rating since 2000, and which genres have shifted? What rating level should we expect next?

Input and preparation. Movies with 20 or more votes were grouped by release month, which gave 312 monthly average ratings (median 72 movies per month) with no missing months. The newest 12 months have too few qualifying movies to model, so we show them but do not model them. That leaves 300 points, from January 2000 to December 2024. We did not run a yearly analysis because our sample holds only about 600 releases per year.

Method and settings. We used Python's statsmodels package [8]. A 12-period additive decomposition separates the series into trend, seasonal and residual parts [9], and an Augmented Dickey-Fuller test [10] checks stationarity. We compared four models: a naive baseline (the mean of the last 12 months), SARIMA with orders chosen by AIC over a small grid, Holt-Winters [11, 12], and damped Holt without seasonality. We trained on January 2000 to December 2022 (276 months), tested on the last 24 months in time order, and scored the results with MAE and RMSE [9]. Steps 1 to 7 are in Review2_TimeSeries.ipynb.



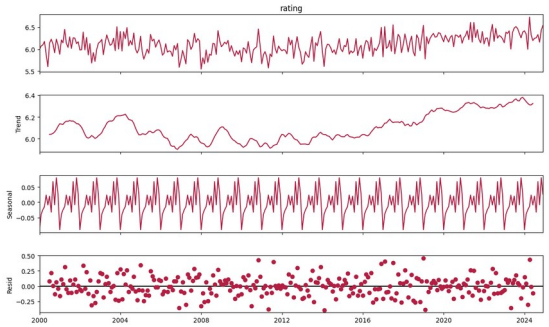


Figure 7. (a) Monthly average rating and popularity with a 12-month mean; the grey band marks recent months not modelled. (b) Additive decomposition of the monthly average rating into trend, seasonal and residual parts. Tool: Python (Matplotlib/statsmodels). Source: Review2_TimeSeries.ipynb, Steps 2 and 3.

Findings. The average rating was flat near 6.06 in the 2000s (6.058) and the 2010s (6.057), then rose to 6.31 for 2020 to 2024 (Figure 7a). Trend strength is 0.41 and seasonal strength only 0.07 (Figure 7b). Average rating by release month only ranges from 6.02 (January) to 6.20 (November), which is not a useful lever. The ADF test ($p = 0.0008$) shows the series is stationary, so no differencing was needed, and the AIC search chose SARIMA(1,0,1) with no seasonal term. The genre mix has also changed (Figure 8). Comparing 2000 to 2004 with 2020 to 2024, the share of Horror grew by 3.8 points and Thriller by 3.3, while Drama fell by 7.2, Romance by 4.1, Action by 3.5 and Comedy by 1.8. A movie can have several genres, so the shares do not add up to 100%. They describe our sample only.

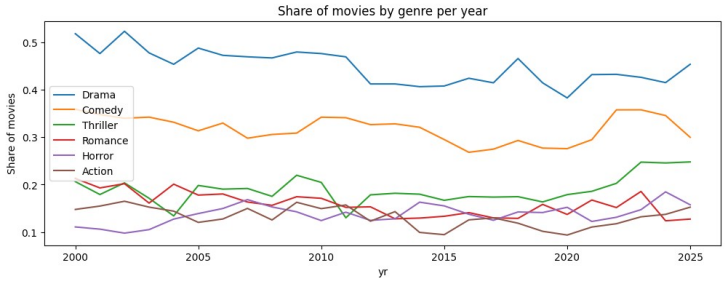


Figure 8. Share of sampled movies in each of the six largest genres by release year. Tool: Python (Matplotlib/statsmodels). Source: Review2_TimeSeries.ipynb, Step 2.

Table 7. Forecast evaluation on the last 24 months (lower is better).

Model	MAE	RMSE	RMSE vs naive
Holt, damped, no season (best)	**0.1252**	**0.1621**	**-6.3%**
SARIMA(1,0,1), no seasonal terms	0.1308	0.1645	-4.9%
Holt-Winters (with seasonality)	0.1326	0.1693	-2.1%
Naive (last 12-month mean)	0.1407	0.1729	0.0%

Evaluation and interpretation. Damped Holt performs best, but it beats the naive model by only about 6%. Ratings are nearly constant, and a more complicated model does not help much (Figure 9a). The Ljung-Box test [13] on the residuals ($p = 0.18$) shows no leftover pattern. For the 12 months we did not model (January to December 2025), the forecast averages 6.36 with a prediction band of about 6.0 to 6.7. For comparison, the last 12 modelled months averaged 6.33 with a band of about 6.1 to 6.5 (Figure 9b). The 2025 movies with 20 or more votes so far average 6.46, which is inside the band but above the point forecast. The limits are that we have only 300 monthly points, some months have few rated movies (as low as 7), and the Holt band is approximate. Read the forecast as ‘about the same level’, not as an exact number.

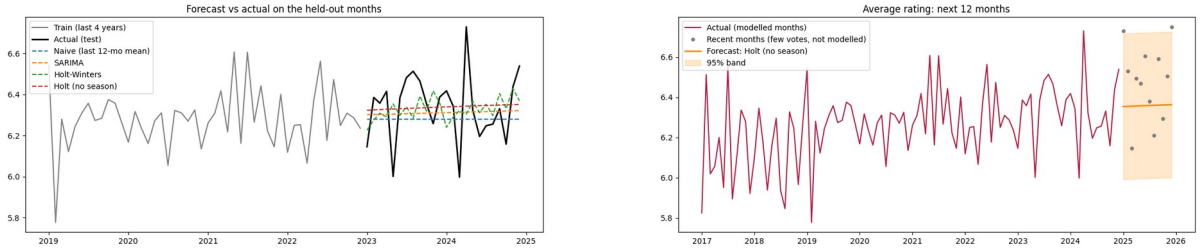


Figure 9. (a) Forecast against actual on the held-out months (January 2023 to December 2024). (b) Forecast of the 2025 average rating (damped Holt) with a 95% band; grey points are months not modelled. Tool: Python (Matplotlib/statsmodels). Source: Review2_TimeSeries.ipynb, Steps 5 and 6.

5.2 Method 2: Text mining of plot overviews

Business question. Which words, themes and tones in a plot summary go with high or low ratings? **Input and preparation.** We used the 10,078 movies with 20 or more votes (about 50 words of overview each). Text was lowercased, reduced to letters only, and stripped of 356 common words, including “film”, “story” and “life”. For comparing words we built a binary document-term matrix with a 2,000-term vocabulary (words that appear in at least 5 movies). For topic modelling we built a count matrix, also with a 2,000-term vocabulary (words in at least 10 movies). Movies were labelled high-rated (6.5 or more) or not.

Method and settings. We used scikit-learn [5] and TextBlob [14]. For each word we ran a chi-square test comparing the share of high-rated movies containing the word with the share of other movies, with a Bonferroni correction (0.05 / 2,000 = 2.5e-5). Topics came from Latent Dirichlet Allocation [15], and the number of topics (4, 6, 8, 10 or 12) was chosen by mean NPMI coherence [16]. We tested rating differences between topics with Kruskal-Wallis and chi-square tests [17]. Sentiment polarity (-1 to +1) was scored with TextBlob. The code is in Text_Mining_Revised.ipynb.

Words. Of the 2,000 words tested, 19 are significantly associated with rating (Figure 10). High-rated movies use more documentary (3.8% vs 1.0%), war (5.0% vs 2.8%), journey, history, music, footage (1.3% vs 0.3%) and interviews (1.2% vs 0.2%). Lower-rated plots use more killer (1.4% vs 3.0%), couple (1.8% vs 3.5%), night (2.7% vs 4.4%), boyfriend (0.8% vs 2.1%), serial and horror. The effect sizes are small, up to 0.10 (phi). The most common phrases are high school (235 movies), best friend, small town and falls love.

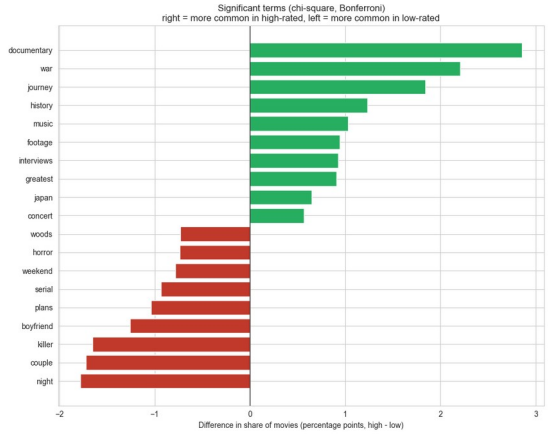


Figure 10. Significant words: right of zero = more common in high-rated plots, left = more common in lower-rated plots (percentage points). Tool: Python (Matplotlib/scikit-learn). Source: Text_Mining_Revised.ipynb, Section 3.

Topics and evaluation. Four topics gave the highest mean coherence (0.034, against 0.028, 0.023, 0.032 and 0.021 for 6, 8, 10 and 12 topics), so we kept four. Absolute coherence is low and the topics are very general. By topic, coherence is 0.008 for Youth, war and journeys, 0.029 for Work, power and adult life, 0.032 for Crime and murder and 0.066 for Love and family. The topic names are our own, based on the top words (Table 8). Rating varies with a movie’s dominant topic (Kruskal-Wallis H = 115.1, p = 8.5e-25; chi-square p = 1.6e-21), although the differences are modest: work and power movies rate 6.31 (46.9% high-rated) and crime and murder movies rate 6.01 (34.7% high-rated; Figure 11).

Table 8. LDA topics with top words, size and rating (movies with 20 or more votes).

Topic (team label)	Top words	Movies	Avg rating	High-rated	Coherence
1. Youth, war and journeys	school, high, war, team, children, fight, parents, journey	2,015	6.16	43.8%	0.008
2. Love and family	love, family, girl, father, mother, past, friends, town	2,905	6.12	36.2%	0.066
3. Work, power and adult life	night, women, game, lives, power, career, husband, company	1,693	6.31	46.9%	0.029

Topic (team label)	Top words	Movies	Avg rating	High-rated	Coherence
4. Crime and murder	house, police, murder, death, killer, city, crime, secret	3,465	6.01	34.7%	0.032

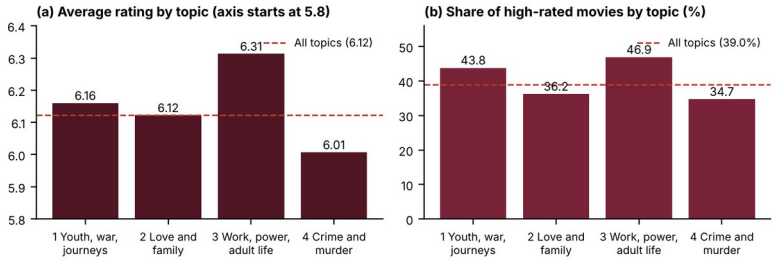


Figure 11. Average rating (left, axis starts at 5.8) and share of high-rated movies (right) by dominant topic. Tool: Python (Matplotlib/scikit-learn). Source: Text_Mining_Revised.ipynb, Section 6.

Sentiment. Plot tone is only weakly related to rating (Spearman r = 0.083, p = 5.5e-17; subjectivity r = -0.035). The difference is statistically significant but small: mean polarity is 0.065 for high-rated movies against 0.036 for the rest (Mann-Whitney p = 1.0e-9; Figure 12), and slightly more summaries are positive for high-rated movies (49.0% vs 44.8%). Horror (-0.065) and Thriller (-0.031) have the most negative summaries, and Romance (+0.136) and Family (+0.103) the most positive. This measures the tone of the writing, not audience opinion. The limits are short texts, no lemmatisation (so ‘falls’ and ‘fall’ count as different words), a lexicon-based sentiment score, and topics that rely on human labels.

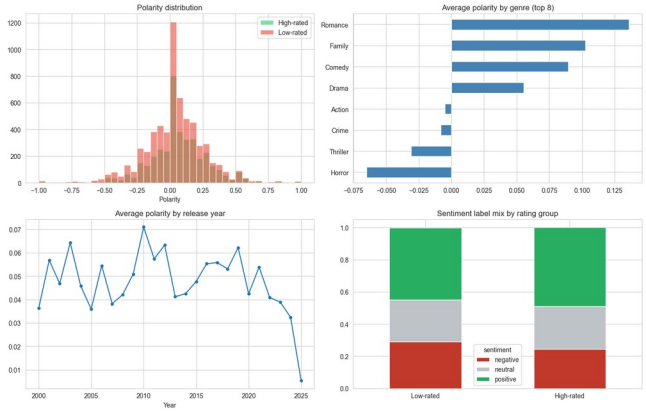


Figure 12. Sentiment of plot summaries: polarity distribution, polarity by genre and year, and sentiment mix by rating group. Tool: Python (Matplotlib/scikit-learn). Source: Text_Mining_Revised.ipynb, Section 7.

6. Comparative Analysis and Business Impact

6.1 Comparison of the two Unit 3 methods

Table 9. Comparison of time-series analysis and text mining.

Aspect	Method 1: Time series	Method 2: Text mining
Question	How do ratings and genre mix change over time; what level next?	Which words and themes go with ratings?
Input	300 monthly average ratings; yearly genre shares	10,078 plot summaries with rating labels
Output	Trend, seasonality, 12-month forecast, genre-share shifts	19 significant words, 4 topics, sentiment scores
Validation	Chronological hold-out (24 months), MAE and RMSE, Ljung-Box test	Chi-square (Bonferroni), coherence, Kruskal-Wallis, Spearman
Strengths	Shows a clear change after 2019; honest baseline comparison	Explains ratings in plain words; links content to rating

Aspect	Method 1: Time series	Method 2: Text mining
Limitations	Only 300 points; forecast barely beats naive; sampled data	Small effects; rough topics; crude sentiment

The two methods cannot be compared on metrics, since forecast error in rating points, p-values and coherence measure different things, so we compare their findings. They work well together. Horror and Thriller show up in both: they have the lowest ratings (5.36 and 5.81) and the biggest gains in share (+3.8 and +3.3 points). The text mining adds the content view: crime and murder plots have the lowest rating (6.01), and Horror and Thriller summaries are the most negative. The overall level has been stable at around 6.3 since 2020, which fits the modest differences between topics, because no single topic shifts the overall level much. Together with the low correlations (0.11 to 0.23), this suggests that genre and theme matter more than any single number.

6.2 Comparison with existing studies

Table 10. Current project compared with the prior studies of Section 1.2.

Criterion	Sharda and Delen [2]	Asur and Huberman [3]	Current project
Data and setting	Movies with pre-release attributes; target is a box-office category	Social-media posts about movies; target is opening-weekend revenue	Self-collected TMDB data, 15,386 movies, 2000 to 2025; target is audience rating
Methods and validation	Neural-network classification, held-out evaluation	Regression on posting rate and sentiment	Logistic Regression, Random Forest, XGBoost with 5-fold CV and held-out test; time series; text mining
Comparable findings / limits	Pre-release attributes carry predictive information; needs star, screen and release data	Social signals carry information; needs external social-media data	Content alone gives AUC 0.785, engagement lifts it to 0.836; no revenue, budget or social data

We cannot compare numbers directly with these studies, because they predict revenue or revenue classes using different data and metrics, while we predict audience rating from TMDB metadata. What we add is a reproducible dataset that is not biased toward popular titles, a split between pre-release and post-release predictability, and a link between ratings, trends and plot text. We do not claim to be better than earlier work. Our limits are the lack of revenue and budget data and the small sample.

6.3 KPI-based business impact and scenarios

Suppose a team evaluates 1,000 candidate titles, of which 39% (390) are high-rated, as in our test data. If it picks titles with the content-only model (set A: precision 0.720, recall 0.527), the model flags about 285 titles. About 205 of them (72%) will be high-rated, while only about 111 (39%) of 285 randomly chosen titles would be. The model would also leave out about 184 of the 390 high-rated titles (47%). Table 11 gives a worst, base and best case. We count benefits in high-rated titles because no cost or revenue data is available.

Table 11. Scenario: picking 285 of 1,000 candidate titles with the screening model (illustrative; assumptions stated).

Scenario	Assumption	High-rated share of picks	High-rated titles in 285 picks	Gain over random picks (111)
Worst	Precision falls to 0.60 on new titles (assumed)	60%	171	+60
Base	**Test precision of set A: 0.720**	**72%**	**205**	**+94**
Best	Test precision of set B: 0.747 (needs early votes)	75%	213	+102

The test set supplies the precision, recall and base rate. The scenarios, the 1,000-title pool and the worst-case precision are our assumptions. A money estimate would need cost and revenue per title, which we do not have. For illustration, if a title costs ₹1 lakh to acquire and a high-rated title is worth ₹5 lakh in engagement, the model's 72% precision means about 205 of the 285 flagged titles would pay off, compared with 39% when

screening without the model. We also assume the future title mix is similar to 2000 to 2025 and that a rating of 6.5 or above is a good sign of success. Section 9 prioritises the recommendations.

7. Interactive Application, Figure Index and Reproducibility

7.1 Mandatory dashboard

The dashboard is a Streamlit [18] and Plotly application (dashboard/app.py). It reads the cleaned data (data/processed) and the notebook outputs (data/ts, data/text) as CSV files. To run it, use pip install -r requirements.txt from the repository root, then streamlit run dashboard/app.py, and open http://localhost:8501. Public link: https://tmdb-movie-analyticss.streamlit.app/ The tabs are Overview, EDA, Time series, Text mining, Model results and Insights.

Data flow. The notebooks write aggregated CSV files, and the app loads them with caching. Everything that depends on tmdb_clean.csv and text_movie_level.csv is calculated on the fly with the filters (release year, genre, language, minimum votes): KPIs, genre and language charts, topic shares and the movie explorer. The forecast, model comparison, seasonality and genre-share files (only the year range applies) are precomputed, as are the Review 1 model results. Figure 13 shows the main and filtered states.

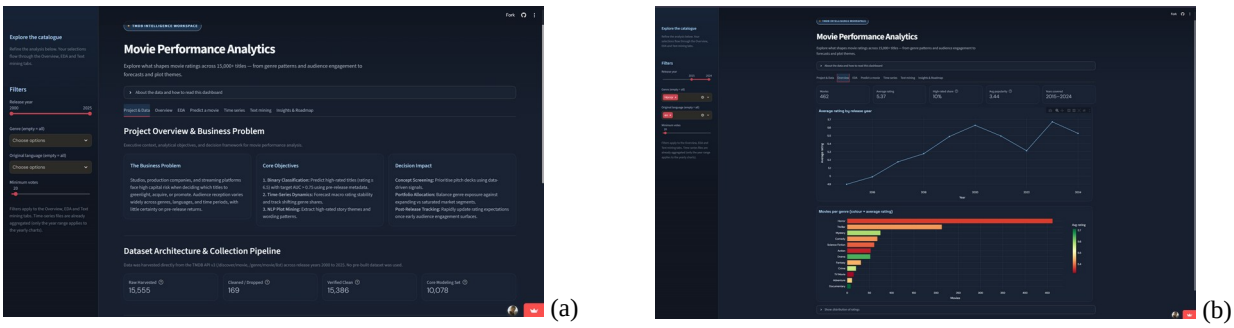


Figure 13. (a) Dashboard main state. (b) Dashboard filtered state, showing that charts and KPIs change. Source: dashboard/app.py.

Tests and limitations. We ran automated smoke tests with Streamlit's AppTest, using copies of the data, changed filters, and no input data at all. No exceptions occurred, and a warning appeared for the missing file. Known weak spots: the time-series charts do not respond to genre or language filters, the model tab is fixed, and a film with two genres is counted once in each genre when averaging.

7.2 Figure index and reproduction map

Table 12. Figure index (each figure is embedded once in Sections 1 to 7; extra figures are in Appendix B).

Figure	Title / tool	Section and key purpose	Source notebook / file
Fig. 1	Workflow diagram, Python	1.1, how the units connect	reports/figures/workflow
Fig. 2, 3	Genre and language; rating by decade, Python	3, genre drives rating	Review1_TMDB_Final.ipynb, EDA
Fig. 4 to 6	Model AUC, confusion/ROC, importance, Python	4, model choice and drivers	Review1_TMDB_Final.ipynb, models
Fig. 7 to 9	Time-series charts, Python	5.1, trend, genre shift, forecast	Review2_TimeSeries.ipynb, Steps 2 to 6
Fig. 10 to 12	Word, topic and sentiment charts, Python	5.2, themes and tone	Text_Mining_Revised.ipynb, Sections 3 to 7
Fig. 13	Dashboard screenshots	7.1, application states	dashboard/app.py

7.3 Code and environment

Repository: https://github.com/soundarya-satal/BA-Project. The project tree has data/raw, data/processed, data/ts, data/text, notebooks (Review1, Review2_TimeSeries, Text_Mining_Revised), models, dashboard, reports, figures, README.md and requirements.txt. It needs Python 3.10 or higher and these packages: pandas, numpy, requests, python-dotenv, scikit-learn, xgboost, scipy, statsmodels, matplotlib, seaborn, wordcloud, textblob, joblib, plotly and streamlit. To reproduce the work, copy .env.example to .env and enter a TMDB API key, then run the notebooks in order (collection and cleaning, EDA and models, time series, text mining), and finally start the dashboard with streamlit run dashboard/app.py. Secrets are not stored in the repository, and the data licence and attribution are in the README.

8. End-to-End Validation, Traceability and Contributions

8.1 Completion and validity

We implemented everything we set out in scope: data collection and validation, exploratory analysis (Unit 1), prediction (Unit 2), time-series analysis and text mining (Unit 3), and the interactive dashboard. Table 13 links each objective to its evidence.

Table 13. Objective-to-evidence traceability matrix.

Objective / KPI	Analysis	Result	Figure	Recommendation
O1: dataset of 10,000 or more records	Section 2	15,386 clean (10,078 for modelling)	Table 3, Table 5	Keep the year-wise sampling for refreshes
O2: AUC of 0.75 or more	Section 4	AUC 0.785 (set A), 0.836 (set B); baseline 0.50	Figure 4 to Figure 6	R2, R3: screen titles, track early votes
O3: forecast beats naive	Section 5.1	RMSE 0.162 vs 0.173 (6% better); level about 6.3	Figure 7 to Figure 9	R4: plan for steady ratings
O4: tested themes	Section 5.2	19 words; 4 topics; Kruskal-Wallis p = 8.5e-25	Figure 10 to Figure 12	R5: use plot themes as a check
O5: dashboard	Section 7	Six tabs with filters over Units 1 to 3	Figure 13	R1: genre benchmarks

Threats to validity. Internally, ratings from few voters are noisy (the 20-vote rule helps), we use one random split plus cross-validation, and importance does not imply cause. Externally, the data are a sample dominated by English-language films, so the conclusions may not hold for other catalogues or other eras. On construct validity, a rating of 6.5 is only an approximation of success, TMDB popularity reflects recent activity, and TextBlob sentiment measures the tone of the description. The collection relies on a live API that changes over time, so we saved both the raw and the clean files. We have not tested the models for subgroup bias, and this is still open.

8.2 Project tracking and individual work

We managed the work on GitHub Projects, with an assignee, due date, checklist and evidence for each task, and moved tasks through Backlog, To Do, In Progress, Review/Testing and Completed.

Board link: https://github.com/users/soundarya-satal/projects/4/views/1. Review 1 covered issues 1 to 17 (30 September to 4 October 2026). Review 2 added the time-series, text-mining, dashboard and final-report tasks. Individual marks may vary with participation and explanation.

Table 14. Team members, assigned work and evidence.

Team member / register no.	Assigned work	Evidence link / artifact
Soundarya Satalgoan, CB.SC.U4CSE23447	Issues 1 to 4 (problem statement, dataset plan, documentation, API setup; 30 Sep to 2 Oct); time-series analysis (setup, stationarity testing, final forecast); report preparation	Issues 1 to 4; Review2_TimeSeries.ipynb (Sections 1, 3, 6: setup, ADF test, forecast); project documentation
Balaji N, CB.SC.U4CSE23011	Issues 5 to 7 (data collection, cleaning, quality check; 30 Sep to 2 Oct); data preprocessing; dashboard development and deployment	Issues 5 to 7; Review1_TMDB_Final.ipynb (data cleaning); dashboard/app.py (dashboard and deployment)
Parvathy Krishna A, CB.SC.U4CSE23739	Issues 8 to 10 (EDA; 2 to 3 Oct); time-series analysis (exploration, model comparison, evaluation)	Issues 8 to 10; Review1_TMDB_Final.ipynb (EDA); Review2_TimeSeries.ipynb (Sections 2, 4, 5: exploration, forecasting models, evaluation)
Venkata Kanna Bhavan Surya Adapa, CB.SC.U4CSE23467	Issues 11 to 13 (feature engineering, target selection, baseline model; 3 to 4 Oct); chi-square word analysis and statistical validation; dashboard integration	Issues 11 to 13; Review1_TMDB_Final.ipynb (features, baseline); Text_Mining_Revised.ipynb (Section 3: word-rating analysis); dashboard/app.py (Model results tab, model integration)
Dareddy Tejeswara Reddy, CB.SC.U4CSE23614	Issues 14 to 17 (model development, tuning, evaluation and interpretation; 3 to 4 Oct); topic modelling and interpretation	Issues 14 to 17; Text_Mining_Revised.ipynb (Sections 5, 6, 9: LDA topic modelling and interpretation)

8.3 AI use and originality declarations

Generative-AI disclosure. The team used Claude (Anthropic) as a generative-AI assistant to plan the issue list and schedule; to draft and debug Python code for data collection and cleaning, exploration, modelling, time-series analysis, text mining and the Streamlit dashboard; to review the text-mining notebook; and to draft, shorten and reword the slides, the README and this report. Human verification: the team ran all notebooks and the dashboard, every number and figure in this report comes from the notebook outputs, and the team checked and corrected the text.

9. Conclusion, Recommendations and Roadmap

As Table 13 shows, we met the objectives listed in Table 1. We built an independent TMDB movie dataset of 15,386 movies, and genre turned out to be the strongest predictor of rating. From content alone we predicted high-rated movies with an AUC of 0.785, and with early engagement the AUC reached 0.836. Ratings show little seasonality, and the level has been steady since 2020. Plot wording and themes are linked to rating.

Table 15. Prioritised recommendations (by impact and feasibility).

#	Recommendation	Expected KPI effect	Stakeholder	Feasibility
R1	Judge each title against its own genre's usual rating (Documentary 6.89, Animation 6.82, Horror 5.36)	Fairer rating targets	Programming	High (data exist)
R2	Use the content-only model as a first screen before release	72% of flagged titles high-rated vs 39%	Acquisition analytics	Medium
R3	Track early votes and popularity after release and refine with the set B model	AUC 0.785 to 0.836	Content analytics	Medium
R4	Plan for steady ratings (about 6.3) and add quality review for Horror and Thriller	Fewer weak titles in growing genres	Production, programming	High
R5	Check pitches against rated plots with similar themes and wording (crime and murder rates lowest)	Earlier warning on risky themes	Development	Medium

Roadmap. Immediate (0 to 1 month): share the genre benchmark table (R1) and pilot the screening on the team's own catalogue. The dashboard is already public (Section 7.1). Medium term (1 to 6 months): add budget, cast and revenue fields, retrain monthly on refreshed data, and set the decision threshold from the cost of missed titles. Longer term (6 to 12 months): prepare the application for wider use, run a controlled comparison of screened and unscreened picks, and add recommendation features. Success will be measured by the share of high-rated picks (above 60%) and by forecast error compared with the naive model.

Not completed in this scope: subgroup bias testing of the models, association rule mining on genre and theme combinations, lemmatisation and a stronger sentiment model, automatic data refresh, a production-grade deployment, and any monetary impact estimate.

References

[1] The Movie Database (TMDB). 2026. TMDB API v3 documentation. Retrieved October 25, 2026 from https://developer.themoviedb.org/docs

[2] Ramesh Sharda and Dursun Delen. 2006. Predicting box-office success of motion pictures with neural networks. Expert Systems with Applications 30, 2 (Feb. 2006), 243-254.

[3] Sitaram Asur and Bernardo A. Huberman. 2010. Predicting the future with social media. In Proceedings of the 2010 IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology, Vol. 1. IEEE, 492-499.

[4] John D. Hunter. 2007. Matplotlib: A 2D graphics environment. Computing in Science & Engineering 9, 3 (2007), 90-95.

[5] Fabian Pedregosa et al. 2011. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12 (2011), 2825-2830.

[6] Leo Breiman. 2001. Random forests. Machine Learning 45, 1 (2001), 5-32.

[7] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM, 785-794.

[8] Skipper Seabold and Josef Perktold. 2010. Statsmodels: Econometric and statistical modeling with Python. In Proceedings of the 9th Python in Science Conference. 92-96.

[9] Rob J. Hyndman and George Athanasopoulos. 2021. Forecasting: Principles and Practice (3rd ed.). OTexts, Melbourne. https://otexts.com/fpp3/

[10] David A. Dickey and Wayne A. Fuller. 1979. Distribution of the estimators for autoregressive time series with a unit root. J. Amer. Statist. Assoc. 74, 366 (1979), 427-431.

[11] Charles C. Holt. 2004. Forecasting seasonals and trends by exponentially weighted moving averages. International Journal of Forecasting 20, 1 (2004), 5-10.

[12] Peter R. Winters. 1960. Forecasting sales by exponentially weighted moving averages. Management Science 6, 3 (1960), 324-342.

[13] Greta M. Ljung and George E. P. Box. 1978. On a measure of lack of fit in time series models. Biometrika 65, 2 (1978), 297-303.

[14] Steven Loria. TextBlob: Simplified text processing. Retrieved November 2, 2026 from https://textblob.readthedocs.io

[15] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet allocation. Journal of Machine Learning Research 3 (2003), 993-1022.

[16] Gerlof Bouma. 2009. Normalized (pointwise) mutual information in collocation extraction. In Proceedings of the Biennial GSCL Conference 2009. 31-40.

[17] William H. Kruskal and W. Allen Wallis. 1952. Use of ranks in one-criterion variance analysis. J. Amer. Statist. Assoc. 47, 260 (1952), 583-621.

[18] Streamlit Inc. Streamlit documentation. Retrieved November 6, 2026 from https://docs.streamlit.io

Appendix A: Extended Data Dictionary and Validation Records

The table counts genres and languages in the clean dataset (a movie can belong to more than one genre). The field-level data dictionary is in Table 5.

Table 16. Movies per genre (19 genres) and per language (six most common of 89) in tmdb_clean.csv.

Genre	Movies	Genre	Movies	Genre	Movies	Genre	Movies
Drama	6,910	Comedy	4,898	Thriller	2,905	Romance	2,450
Horror	2,117	Action	2,033	Documentary	1,515	Crime	1,510
TV Movie	1,216	Family	1,130	Adventure	1,028	Animation	1,021
Science Fiction	1,006	Mystery	984	Fantasy	963	History	587
Music	518	War	293	Western	99		

Languages: en 8,229, fr 1,138, es 806, ja 751, it 457, de 443. Other validation records: movies per release year, 575 to 598. Summary statistics (mean, median, maximum): popularity 2.77, 1.88, 92.51; vote average 6.00, 6.10, 10.00; vote count 331, 30, 34,747. Sources: Review1_TMDB_Final.ipynb and DATA_DICTIONARY.md.

Appendix B: Additional Figures and Supplementary Analysis

Some supplementary figures that do not appear in the main text are kept in the notebooks and the figures folder: rating against vote count and density plots (Review1_TMDB_Final.ipynb, EDA relationships); rating and release share by calendar month; ACF/PACF plots; residual plots; word clouds; most common words and phrases; coherence against number of topics; and topic shares by year and genre (Text_Mining_Revised.ipynb, Sections 3 to 6). Two of them are shown below.

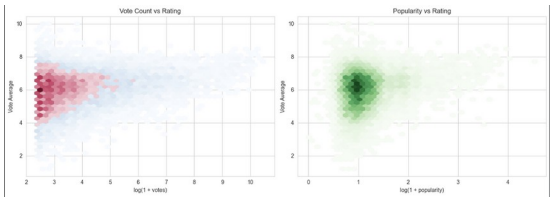


Fig. B1. Rating against vote count and popularity (density plots, log scales).

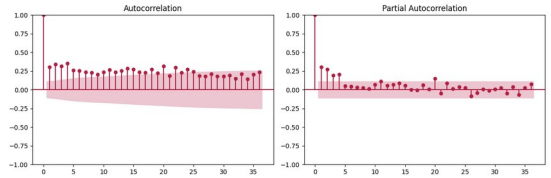


Fig. B2. ACF and PACF of the monthly rating series.

Appendix C: Implementation Details and Dashboard User Guide

Table 17. Key settings used.

Component	Settings
Collection	years 2000 to 2025; 30 random pages per year (seed 42); sort popularity.desc; vote_count >= 10; include_adult = false; retries with back-off; checkpoint file
Models	Random Forest set B: depth 20, min leaf 3; XGBoost set B: depth 3, 400 trees, learning rate 0.05; 5-fold stratified CV, AUC scoring; split 80/20, seed 42
Time series	monthly mean rating, 300 points; decomposition period 12; SARIMA grid p,q in 0 to 2, P,Q in 0 to 1; hold-out 24 months; forecast horizon 12 months
Text mining	2,000-term matrices; min document frequency 5 (words) and 10 (topics); LDA online, 20 iterations, seed 42; K tried: 4, 6, 8, 10, 12; Bonferroni alpha 2.5e-5

Dashboard user guide. Run streamlit run dashboard/app.py. Use the sidebar to change the release-year range, genres, languages and minimum votes, and the Overview, EDA and Text mining tabs update. The Time series tab shows the trend, forecast and model comparison. The Text mining tab has the topics, the words linked to rating, sentiment, and a searchable movie explorer that can be downloaded as CSV. If a file is missing, the app names the notebook that creates it.

Appendix D: Tracking Evidence, Contributions and Declarations

Board evidence (a screenshot of the GitHub Projects board with assignees, due dates and stages) is shown in Fig. D1. Contribution records are in Table 14. The AI-use declaration is in Section 8.3.

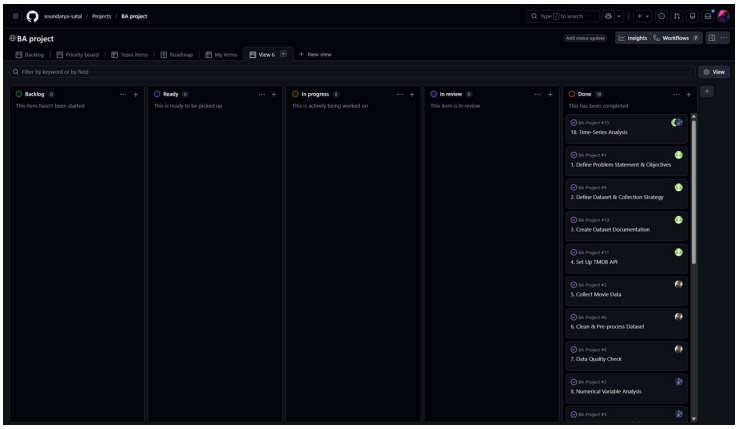


Fig. D1. GitHub Projects board (BA project) with workflow stages and assignees.